

Regression Algorithms for Learning & Their Application on Cryptocurrency Markets

Final Report

Syed Maesum Hussain Zaidi

Undergraduate Final Year Project

BSc Computer Science (Artificial Intelligence)



Supervised By : Xu Feng

Department of Computer Science

Royal Holloway, University of London

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 12622

Student Name: Syed Maesum Hussain Zaidi

Date of Submission: 15 April, 2026

Signature:

A handwritten signature in black ink that reads "Maesum" followed by a long, sweeping diagonal stroke.

Table of Content

Table of Content.....	2
1. Abstract.....	4
2. Project Specification.....	5
2.1 Core Requirements.....	6
2.2 Functional Requirements.....	6
2.3 Software Engineering Requirements.....	6
2.4 Analytical Requirements.....	7
2.5 Extended Features.....	7
2.6 Deliverables Achieved.....	8
3. Introduction.....	8
4. Aims & Objectives.....	10
4.1 Aims.....	11
4.2 Objectives.....	11
5. Background Theory.....	11
5.1 Linear Regression Models.....	12
5.2 Ordinary Least Squares (OLS).....	12
5.3 Ridge Regression.....	13
5.4 Lasso Regression.....	14
5.5 Bias–Variance Trade-off.....	14
5.6 Comparison Table of OLS, Ridge and Lasso Regression.....	14
5.7 Relevance to this Project.....	15
6. Literature Review.....	15
6.1 Regression Based Approaches in Financial Forecasting.....	16
6.2 Fischer and Krauss (2018).....	16
6.3 Patel et al. (2015).....	17
6.4 Key Insights from the Literature.....	17
6.5 Summary Table for the Two Studies Reviewed.....	18
6.6 Relevance to this Project.....	18
7. Methodology.....	19
7.1 Data Collection.....	19
7.2 Data Preprocessing.....	20
7.3 Feature Engineering.....	20
7.4 Algorithm Implementation.....	21
7.5 Model Training and Validation.....	21
7.6 Evaluation Metrics.....	22
7.7 System Workflow.....	23
7.8 Key Design Choices.....	23

7.9 Justification of Technical Decisions.....	24
8. System Design.....	25
8.1 Design Objectives.....	25
8.2 Overall Architecture.....	26
8.3 Main Components of the System.....	26
8.3.1 Configuration Component.....	26
8.3.2 Data Loading Component.....	27
8.3.3 Feature Engineering Component.....	27
8.3.4 Model Implementation Component.....	27
8.3.5 Tuning Component.....	28
8.3.6 Evaluation Component.....	28
8.3.7 Prediction Component.....	28
8.3.8 Plotting Component.....	28
8.3.9 Interface Component.....	29
8.4 Object-Oriented Design.....	29
8.5 Unified Modelling Language (UML) Diagram.....	30
8.6 System Interaction Flow.....	31
8.7 Design Benefits.....	31
9. Implementation.....	32
9.1 Overall Implementation Structure.....	32
9.2 Data Loading and Preprocessing Implementation.....	33
9.3 Feature Engineering Implementation.....	33
9.4 Algorithm Implementations.....	34
9.5 Hyperparameter Tuning Implementation.....	34
9.6 Walk-Forward Evaluation Implementation.....	35
9.7 Prediction Implementation.....	35
9.8 Visualisation Implementation.....	36
9.9 Graphical User Interface (GUI).....	36
9.9.1 GUI Description.....	37
9.9.2 GUI Visuals.....	37
9.10 Implementation Summary.....	42
9.11 Development Process and Challenges.....	42
10. Results.....	43
10.1 Case Study 1: Ethereum (ETH).....	43
10.2 Case Study 2: Chainlink (LINK).....	45
10.3 Case Study 3: HBAR.....	46
10.4 Summary of Results.....	47
11. Discussion.....	48
11.1 Impact of Market Volatility.....	48
11.2 Comparison Between Linear and Non-Linear Models.....	48
11.3 Behaviour of K-Nearest Neighbours.....	49

11.4 Relationship Between MSE and Directional Accuracy.....	49
11.5 Limitations of Neural Networks in This Context.....	50
11.6 Effectiveness of Walk-Forward Validation.....	50
11.7 Overall Interpretation.....	50
12. Evaluation.....	51
12.1 Achievement of Objectives.....	51
12.2 Model Performance.....	51
12.3 Effectiveness of the System.....	52
12.4 Strengths of the Approach.....	52
12.5 Areas for Improvement.....	52
12.6 Summary.....	52
13. Limitations.....	53
13.1 Data Limitations.....	53
13.2 Model Limitations.....	53
13.3 Feature Engineering Limitations.....	53
13.4 Evaluation Limitations.....	54
13.5 System Limitations.....	54
13.6 Summary.....	54
14. Future Work.....	55
14.1 Incorporation of Additional Data Sources.....	55
14.2 Improvement of Neural Network Architecture.....	55
14.3 Expansion of Feature Engineering.....	55
14.4 Alternative Evaluation Strategies.....	56
14.5 Real World Trading Considerations.....	56
14.6 System Improvement.....	56
14.7 Summary.....	56
15. Conclusion.....	57
16. Bibliography.....	58
17. Appendices.....	59
17.1 Appendix A.....	59

1. Abstract

This project primarily investigates the use of regression based machine learning algorithms for predicting cryptocurrency price movements. The aim of this project is to not only understand the theoretical foundations of regression models but also to evaluate their effectiveness when applied to highly volatile financial time-series data such as that of cryptocurrency markets.

The project begins with a detailed exploration of core regression techniques which includes Ordinary Least Squares (OLS), Ridge Regression, and Lasso Regression focusing on their mathematical structure, cost functions, and behaviour under regularisation. This theoretical understanding is supported by a literature review examining how such machine learning algorithms are applied in modern financial forecasting systems.

First, a proof-of-concept model was developed using synthetic data to verify the correctness of the regression implementations. After that, a full prediction system was built using real cryptocurrency data across 25 handpicked different cryptocurrency pairs obtained from Yahoo Finance. The system incorporates feature engineering, walk-forward validation, and multiple regression models implemented from scratch.

The final system further extended the project beyond basic regression by including additional algorithms such as K-Nearest Neighbours, Kernel Ridge Regression, and a feedforward neural network. A graphical user interface (GUI) was also developed to allow users to interact with the model, select datasets, and visualise results in the currency of their choice between United States Dollar (USD) or Great Britain Pound (GBP).

The performance of the algorithms is then evaluated using Mean Squared Error (MSE) on both predicted prices and returns, as well as directional accuracy. The results demonstrate that while regression models provide a useful baseline for financial prediction, their performance is limited by the complexity and non-stationarity of cryptocurrency markets.

2. Project Specification

This section of the report lists the initial requirements of the project based on the original project plan. Then it explains how these requirements were addressed in the final implemented model. It provides a clear overview of the intended objectives and the functionalities which were achieved in the completed project.

2.1 Core Requirements

The project was designed to become a complete machine learning model capable of applying different regression algorithms to real world financial data. In the final implementation, this objective was achieved through the development of a structured and modular system that integrates data collection, preprocessing, modelling, evaluation, and visualisation.

The final model successfully implements regression algorithms to predict next-day cryptocurrency prices using real-world datasets. It supports user interaction through a graphical interface and provides automated evaluation and comparison between multiple different algorithms.

2.2 Functional Requirements

The objectives of this project required several key functionalities. These functionalities have been fully implemented in the final system.

The model loads historical cryptocurrency data using the Yahoo Finance API. Then it processes it to ensure it is fit to use for the model. This includes handling missing values, structuring the dataset, and preparing it for feature generation.

Feature engineering is performed to create meaningful inputs for the algorithms so they can perform better. This includes lag-based features, moving averages, and volatility measures which are commonly used in financial modelling.

The system trains multiple machine learning algorithms on the processed data and evaluates their performance using a walk-forward validation framework. A sliding window approach is used to ensure that the models are trained on recent data thus making the predictions more relevant to current market conditions.

The system generates next-day price predictions and compares the performance of different algorithms. The results are presented in both numerical and graphical formats allowing for clear interpretation.

2.3 Software Engineering Requirements

The project was implemented using an object-oriented design to ensure modularity and maintainability. The system is divided into separate components responsible for data handling, feature engineering, modelling, evaluation, and visualisation.

Each component is implemented in a structured way which allows the system to be easily extended with additional models or features. The use of modular design also improves code readability and maintainability.

A key aspect of the implementation is that the regression algorithms were developed from scratch using NumPy instead of relying on complete implementations from external libraries like sci-kit learn. This demonstrates a deeper understanding of the underlying mathematical concepts and ensures transparency in how the models operate.

2.4 Analytical Requirements

The analytical component of this project combines theoretical understanding with experimental evaluation.

The theoretical foundations of regression algorithms are explored in depth which includes their mathematical formulations and behaviour under regularisation. A literature review was also conducted to understand how similar approaches have been previously applied in financial forecasting before.

The model compares multiple different algorithms including regression based algorithms and alternative approaches such as K-Nearest Neighbours and neural networks. The performance comparison of these algorithms is performed using Mean Squared Error (MSE) on both predicted prices and returns, as well as the directional accuracy.

The results are analysed to identify the strengths and limitations of each model with an emphasis in the context of highly volatile cryptocurrency markets.

2.5 Extended Features

In addition to the core requirements of the project several extended features were implemented during the development of the model to enhance it.

The system supports multiple cryptocurrency pairs rather than being limited to a single dataset. In total, 25 different cryptocurrency pairs are included which are various in their nature. This allows for broader experimentation and analysis. Additionally, Users are able to choose between the United States Dollar (USD) and Great Britain Pound (GBP) when viewing predictions making the system more flexible and user friendly.

Additional algorithms were implemented beyond the original scope which focused on only regression based algorithms. These new algorithms include Kernel Ridge Regression, K-Nearest Neighbours, and a feedforward neural network. These models provide a basis for comparison with traditional regression approaches.

The use of a sliding window validation approach represents an improvement over the initially considered expanding window method, as it ensures that the model focuses on more recent and relevant data. This is key when focusing on the cryptocurrency markets because these markets are fast evolving and in recent times have seen the significant increase in institutional investments which have majorly changed the behavior of the market. Hence, data from years ago is not significantly useful when trying to understand the pattern of the market to make predictions as compared to more recent data.

2.6 Deliverables Achieved

The final project successfully delivers all key components outlined in the initial specification.

A fully functional machine learning model has been developed which includes a graphical user interface that allows users to interact with the model in order to select parameters and visualise results.

The system produces visual outputs such as performance graphs, prediction plots, and comparison tables, which are used to analyse different algorithms performance.

The final report compiles everything together including theoretical foundations of the algorithms, implementation details, experimental results, and evaluation of the models.

UML diagrams and system visualisations are included as additional supporting material which illustrate the design and functionalities of the model.

3. Introduction

Financial time-series prediction is widely recognised as a challenging problem due to the presence of noise, volatility, and non-stationary behaviour. In particular, cryptocurrency markets show extreme price fluctuations influenced by a combination of market sentiment, macroeconomic events, and speculative trading activity. These characteristics make accurate prediction difficult, but they also provide a meaningful environment in which machine learning models can be tested and evaluated.

Regression algorithms form one of the most fundamental approaches in machine learning for modelling relationships between variables. They provide a mathematically interpretable framework that allows us to estimate how input features influence a continuous output variable. Regression models due to their simplicity and transparency are often used as a baseline before moving to more complex algorithms.

The initial phase of this project focused on building a strong theoretical understanding of regression techniques, particularly Ordinary Least Squares, Ridge Regression, and Lasso Regression. These models differ primarily in how they handle overfitting and multicollinearity through regularisation. Ridge Regression introduces an L2 penalty to stabilize coefficients, while Lasso Regression uses an L1 penalty that can also perform feature selection.

Following this theoretical groundwork, the project transitioned into implementation. A proof-of-concept model was created using synthetic data to verify that the regression algorithms behaved as expected. Once the proof-of-concept was validated, the model was extended to real-world cryptocurrency data, specifically BTC-USD, where the models were applied to predict next day prices.

As the project progressed, the focus expanded beyond basic regression models. Additional algorithms such as K-Nearest Neighbours, Kernel Ridge Regression, and a feedforward neural network were implemented to allow for performance comparison. As the number of algorithms expanded, the number of datasets was also expanded by moving from only BTC-USD to 25 different handpicked cryptocurrency pairs. A key improvement made during development was the use of walk-forward validation. Initially, an expanding window approach was considered where the training data continuously increases over time. However, due to the fast-changing nature of cryptocurrency markets, a sliding window approach was adopted instead. This ensures that the model is trained on more recent data which is more relevant for prediction, while older data that may no longer reflect current

market conditions is excluded in the next day prediction. An important thing throughout the project was that all algorithms were implemented from scratch using the NumPy library instead of importing existing complete implementations.

A graphical user interface was developed to improve usability and presentation of the model by allowing users to select cryptocurrency pairs, adjust parameters, and visualise results through both the next day prediction and performance comparison graphs and tables. Furthermore, the users are allowed to pick between USD or GBP to visualize the price predictions.

Overall, this project combines theoretical understanding with practical implementation to explore how regression based learning methods perform in the context of cryptocurrency prediction. It highlights both the strengths and limitations of these models when applied to complex financial data.

4. Aims & Objectives

The aim of this project is to investigate how regression based machine learning algorithms perform when applied to the prediction of cryptocurrency price movements. The focus is not only on building a working prediction system, but also on understanding the behaviour, strengths, and limitations of these models in the context of highly volatile financial time-series data.

In addition to prediction accuracy, the project also aims to explore how different modelling choices such as regularisation techniques and validation strategies influence performance.

4.1 Aims

- To develop a strong theoretical understanding of regression algorithms including Ordinary Least Squares (OLS), Ridge Regression, and Lasso Regression.
- To investigate how regularisation techniques affect model stability and generalisation in noisy financial datasets.
- To analyse the effectiveness of regression models for predicting next-day cryptocurrency prices.
- To compare regression based models with alternative machine learning approaches such as K-Nearest Neighbours and neural networks.
- To understand the impact of validation strategies particularly walk-forward validation on model evaluation.

4.2 Objectives

In addition to these aims, the project also sets some supporting objectives which are as follows:

- To implement regression algorithms from scratch using NumPy to ensure a clear understanding of their internal workings.
- To validate the correctness of the implementations using a proof-of-concept model on synthetic data.
- To apply the models to real-world cryptocurrency datasets and generate next-day price predictions.
- To design and apply a sliding window walk-forward validation framework for realistic time-series evaluation.
- To evaluate model performance using Mean Squared Error (MSE) on predicted prices and returns, as well as directional accuracy.
- To analyse and compare the performance of multiple algorithms across different datasets.
- To interpret the results and identify the strengths and limitations of each modelling approach.

5. Background Theory

This section of the report focuses on the theoretical foundations of the regression algorithms used in this project. Regression algorithms form a core part of supervised learning. They are widely used to model relationships between input variables and continuous outputs. In the context of this project, they are used to estimate and predict cryptocurrency price movements.

The theoretical understanding developed in this section is based on key machine learning texts, particularly those by Bishop (2006) and Hastie, Tibshirani, and Friedman (2009) which describe regression as one of the fundamental building blocks of statistical learning.

5.1 Linear Regression Models

Regression aims to model the relationship between a set of input variables and an output variable. The general assumption is that the output can be expressed as a function of the inputs along with some random noise.

In matrix form, the linear regression model is written as:

$$y = X\beta + \varepsilon$$

Where:

- y = $N \times 1$ vector of outputs received
- X = $N \times p$ design matrix containing input features
- β = $p \times 1$ coefficient vector
- ε = vector of random noise
- p = the number of input variables

The goal of the model is to estimate the coefficient vector β such that the predicted values are as close as possible to the actual observed values.

5.2 Ordinary Least Squares (OLS)

Ordinary Least Squares (OLS) is the simplest and most commonly used regression method. It works by minimising the sum of squared differences between the predicted values and the actual values.

The objective function is defined as:

$$\min_{\beta} \|y - X\beta\|^2$$

The closed-form solution for OLS is:

$$\beta = (X^T X)^{-1} X^T y$$

- y = Vector of actual (observed) outputs
- X = Design matrix or feature matrix
- β = Vector of model parameters
- $()^T$ = Transpose operator
- $()^{-1}$ = Inverse

This solution provides an exact estimate of the coefficients under the assumption that the matrix $(X^T X)$ is invertible. OLS is widely used because of its simplicity and interpretability. However, it can become unstable when the input variables are highly correlated or when the dataset contains a high level of noise. These issues are particularly common in financial datasets where many indicators capture overlapping information (Hastie et al., 2009).

5.3 Ridge Regression

Ridge Regression extends OLS by introducing the L2 regularisation term that penalises large coefficient values. This helps to reduce overfitting and improves the stability of the model.

The Ridge objective function is:

$$\min_{\beta} (\|y - X\beta\|^2 + \lambda \|\beta\|^2)$$

The corresponding closed-form solution is:

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

Where:

- λ = Regularization parameter
- I = Identity Matrix

By adding the term λI , the matrix becomes more stable and easier to invert. This reduces the magnitude of the coefficients and prevents the model from fitting noise in the data. Ridge Regression is particularly useful in cases where multicollinearity exists as it distributes the influence across correlated variables rather than allowing extreme coefficient values (Bishop, 2006).

5.4 Lasso Regression

Lasso Regression introduces an alternative form of regularisation using an L1 penalty. This leads to a different type of behaviour compared to Ridge Regression.

The objective function is:

$$\min_{\beta} (\|y - X\beta\|^2 + \lambda \|\beta\|_1)$$

Unlike Ridge Regression, Lasso does not have a closed-form solution due to the absolute value term. Instead, it is typically solved using iterative optimisation methods such as Coordinate Descent. Coordinate Descent is an optimization method that updates one coefficient at a time while keeping the others fixed, thus repeatedly cycling through all coefficients until the model converges to a minimum. A key property of Lasso Regression is that it can shrink some coefficients exactly to zero. This effectively performs feature selection which allows the model to identify the most important variables in the dataset (Hastie et al., 2009).

5.5 Bias–Variance Trade-off

An important concept in regression modelling is the trade-off between bias and variance.

- **Bias** refers to the error introduced by simplifying assumptions in the model
- **Variance** refers to how sensitive the model is to changes in the training data

OLS in general has low bias but can have high variance, especially in noisy datasets. Ridge and Lasso introduce regularisation, which slightly increases bias but reduces variance, leading to better generalisation on unseen data.

This balance is particularly important in financial prediction tasks where overfitting to historical data can result in poor predictive performance.

5.6 Comparison Table of OLS, Ridge and Lasso Regression

Features	OLS	Ridge	Lasso
Objective Function	$\min_{\beta} \ y - X\beta\ ^2$	$\min_{\beta} (\ y - X\beta\ ^2 + \lambda \ \beta\ ^2)$	$\min_{\beta} (\ y - X\beta\ ^2 + \lambda \ \beta\ _1)$
Solution	$\beta = (X^T X)^{-1} X^T y$ (Closed Form)	$\beta = (X^T X + \lambda I)^{-1} X^T y$ (Closed Form)	Coordinate Descent (Iterative)
Regularization Effect	None	Shrinks Coefficients	Shrinks and sets some to zero
Bias-Variance	Low Bias High Variance	Slight Bias Reduced Variance	Sparse model Reduced Variance
Interpretability	Uses all Features	Uses all Features	Selects Key Features
When Useful	Small Clean Datasets	Correlated Predictors	High Dimension Sparse Data

5.7 Relevance to this Project

The regression models described in this section form the foundation of the system developed in this project. Their mathematical simplicity makes them suitable for implementation from scratch, while their interpretability allows for a clearer understanding of how different features influence predictions.

However, due to the complexity and volatility of cryptocurrency markets, these models are also compared with more flexible algorithms such as K-Nearest Neighbours and neural networks in later sections of the report. This allows for a more complete evaluation of their effectiveness in real-world prediction tasks such as cryptocurrency price prediction.

6. Literature Review

Financial market prediction has been widely studied in both traditional finance and machine learning research. Over time, regression based methods have formed the foundation for many predictive models even as more advanced techniques have been introduced. This section of the report looks at some key studies that apply regression principles and machine learning techniques to financial forecasting thus providing context for the approaches used in this project.

6.1 Regression Based Approaches in Financial Forecasting

Historically, Regression models have been one of the most important tools for financial prediction. They allow for the estimation of future values based on historical data by providing a statistical framework for modelling relationships between variables.

Foundational texts such as Bishop (2006) and Hastie, Tibshirani, and Friedman (2009) highlight that regression methods are not only useful for prediction but also serve as the basis for many modern machine learning algorithms. Parameter estimation, regularisation, and error minimisation are some of the concepts that continue to play a central role in more complex models.

Regression models in financial contexts are often used to predict price movements, returns, or trends based on technical indicators and historical patterns. However, the effectiveness displayed by these models depends heavily on feature representation and their ability to generalise to unseen data.

6.2 Fischer and Krauss (2018)

The use of Long Short-Term Memory (LSTM) networks for predicting stock price movements in the S&P 500 was investigated in Fischer and Krauss (2018). Their approach focused on training models on sequences of historical returns and then using these sequences to predict whether a stock would outperform the market.

Although LSTM networks are considered a deep learning technique, their internal operations still rely on principles similar to regression. Before applying non-linear activation functions, each layer applies weighted transformations to the input data similar to linear regression models.

The results of their study showed that LSTM models outperformed traditional approaches such as logistic regression and random forests in predicting market movements. This demonstrates the advantage of capturing temporal dependencies in financial data.

An important takeaway however from this study is that even complex models are built on regression based concepts. This supports the idea that understanding regression provides a strong foundation for developing more advanced prediction systems.

6.3 Patel et al. (2015)

The use of machine learning techniques for predicting stock market movements using different types of input data was explored by Patel, Shah, Thakkar, and Kotecha (2015). Their study compared models trained on raw technical indicators with those trained on trend-based representations of the data.

Their findings showed that feature engineering plays a critical role in model performance. In particular, transforming raw data into more meaningful representations significantly improved prediction accuracy across multiple algorithms.

This insight is directly relevant to this project as this project uses feature engineering to create lag-based features, moving averages, and volatility measures. These features help the regression models capture patterns in cryptocurrency price movements more effectively.

6.4 Key Insights from the Literature

Across the studies reviewed, several common themes emerge:

- Even in advanced machine learning models regression principles remain fundamental.
- Feature engineering has a significant impact on predictive performance.
- Financial data is highly noisy and requires careful validation techniques.
- More complex models can improve performance however they also introduce additional complexity and reduced interpretability.

These insights directly influenced the design decisions made in this project. In particular, the importance of feature engineering and the need for robust validation methods led to the use of a sliding window walk-forward validation approach.

6.5 Summary Table for the Two Studies Reviewed

Both works look into the predictive capabilities of regression related models within financial markets but differ in their methodological approach. Fischer and Krauss extend regression into a temporal sequence-learning framework using LSTM networks, while Patel, Shah, Thakkar, and Kotecha examine feature based transformations and ensemble methods to improve prediction accuracy.

Aspect	Fischer & Krauss LSTM Regression for Financial Time Series	Patel, Shah, Thakkar, & Kotecha Trend Based Regression & Data Preparation
Dataset	S&P 500 daily returns (1992–2015)	Indian stocks and indices (2003–2012)
Model Type	LSTM (sequential regression)	ANN, SVM, RF, Naïve Bayes
Key Contribution	Demonstrates effectiveness of memory based models for market sequences	Shows impact of trend based data encoding
Connection to Regression	Regularised temporal modelling	Feature engineering and ensemble regression
Core Outcome	Outperformed benchmarks in accuracy and returns	Improved results through TDDP preprocessing

6.6 Relevance to this Project

The literature reviewed highlights that while advanced models such as neural networks can achieve strong predictive performance, regression based models remain an essential starting point due to their simplicity, interpretability, and mathematical transparency.

This project builds on these ideas by first implementing regression models from scratch and then extending the system to include more complex algorithms. This allows for a direct comparison between simpler and more advanced methods in the context of cryptocurrency prediction.

By combining insights from existing research with practical implementation, the project aims to evaluate how well regression based approaches perform in highly volatile markets and whether they provide a reliable baseline for financial forecasting.

7. Methodology

This section of the report describes the overall approach taken to design, build and evaluate the prediction system. It explains how the data was collected and processed, how features were generated, how the models were trained, and how their performance was evaluated.

The main purpose of this methodology is to ensure that the system is both technically correct and suitable for financial time-series prediction where data is highly volatile and constantly changing.

7.1 Data Collection

The data used in this project is obtained from Yahoo Finance using the `yfinance` python library. This library allows access to historical financial data including but not limited to daily opens, highs, lows, closes, and volume values.

The primary focus of the project is on cryptocurrency data. Initial testing was conducted using a single dataset (BTC-USD). However, the final system was extended to support 25 different cryptocurrency pairs. These pairs were selected to represent a variety of market behaviours, including both high-cap and lower-cap cryptocurrencies. (The full list of these cryptocurrencies can be found in Appendix A)

The time range of the data depends on the availability of each asset but typically spans multiple years in order to provide enough data for training and evaluation. The 25 pairs of cryptocurrency used in this project have at least 3 years of historical data available on `yfinance`. The reason for this is that the model can be run on historical data between 3-10 years in the past from the date of last close so any cryptocurrency pair that does not have 3 years of data would not be applicable to use in the model.

7.2 Data Preprocessing

The data must be processed to make sure it is clean and consistent before it can be used for modelling.

The preprocessing stage includes:

- Removing missing values where necessary
- Ensuring the data is sorted chronologically
- Converting the dataset into a structured format suitable for modelling

These steps are important to make sure that the models are trained on reliable data since financial data can contain irregularities such as missing days or incomplete records.

In addition, the target variable is defined as the next-day closing price. This is achieved by shifting the closing price column forward by one time step so that each row of input features corresponds to the price that occurs on the following day.

7.3 Feature Engineering

Feature engineering plays a key role in improving model performance. Instead of just relying only on raw price data, additional features are created to help the algorithms capture patterns in the price movement more effectively.

The main types of features used in this project include:

- **Lag features:** Previous values of the closing price are used as inputs
- **Moving averages:** Average prices over a rolling window to capture trends
- **Volatility measures:** Indicators that reflect how much the price fluctuates over time

These features are commonly used in financial modelling and are supported by the literature reviewed earlier. In particular, the findings of Patel et al. (2015) which highlight the importance of transforming raw data into more meaningful representations.

7.4 Algorithm Implementation

A key aspect of this project is that all the algorithms are implemented from scratch including both the regression based and non regression based.

In the first stage the algorithms implemented were:

- Ordinary Least Square (OLS) Regression
- Ridge Regression
- Lasso Regression

Implementing these models manually using NumPy ensured a clear understanding of how they operate internally rather than relying on importing black-box implementations from external libraries of python like sci-kit learn.

In the later stage of the project three more algorithms were implemented:

- Kernel Ridge Regression
- K-Nearest Neighbours (KNN)
- Feedforward Neural Network

These models were included to evaluate the comparison between simple regression based approaches and more flexible or non-linear methods.

7.5 Model Training and Validation

A major challenge in financial prediction is ensuring that the evaluation process reflects real world conditions. Standard random train-test splits are not suitable for time-series data because they can introduce look-ahead bias. In order to solve this problem a walk-forward validation approach was used.

During the initial stages of the project, an expanding window implementation was considered where the training data increases over time. However, that was then changed to a sliding window implementation because the cryptocurrency market is a relatively new market which is fast evolving and institutional investments have only begun in recent events. In addition, governments around the world have started to adapt to cryptocurrency by properly implementing laws and regulations on them which have completely changed the market behavior in recent years due to which the market data from a decade ago would not be as relevant to the current market behaviors and conditions as compared to the data of recent years.

The sliding window approach works as follows:

- A fixed-size training window is selected
- The model is trained on this window
- It is then tested on the next period
- The window is moved forward and the process is repeated

This makes sure that the model is always trained on recent data which makes the evaluation more realistic and better suited to non-stationary markets.

7.6 Evaluation Metrics

The performance of the different algorithms is evaluated using three main metrics:

- Mean Squared Error (MSE) on predicted prices
- Mean Squared Error (MSE) on predicted returns
- Directional accuracy

MSE is an evaluation metric which measures how close the predicted values are to the actual values. It is applied separately to price predictions and return predictions in order to capture different aspects of the model's performance.

Directional Accuracy is a measure of how often the algorithm correctly predicts the direction of the price movement (whether the price will go up or down). This is specifically important for financial market applications where just being able to predict the correct direction can at times be more useful to being able to predict the actual value.

7.7 System Workflow

The overall workflow of the system can be summarised as follows:

1. Load historical cryptocurrency data from Yahoo Finance
2. Preprocess the data and define the target variable
3. Generate features from the raw data
4. Train multiple algorithms using the sliding window approach
5. Evaluate algorithm performance using defined metrics
6. Generate next-day predictions
7. Visualise results using graphs and comparison tables

This workflow is implemented in a modular way using object oriented programming principles which allows different components of the system to be reused or extended.

7.8 Key Design Choices

Several key design decisions were made during the development of the system:

- Use of sliding window validation instead of expanding window to focus on recent data
- Implementation of models from scratch to ensure understanding and transparency
- Inclusion of multiple different cryptocurrency datasets to test generalisability
- Comparison with non-linear models to evaluate the limitations of regression
- A GUI which not only allows the user to choose the cryptocurrency pair and the amount of data to use for the model but also gives them the choice to choose the currency between USD and GBP to visualize their results

These choices were influenced by theoretical understanding, insights from the literature review and personal preferences. These design decisions directly reflect the theoretical principles discussed earlier and aim to ensure that the system performs reliably under realistic market conditions.

7.9 Justification of Technical Decisions

Throughout the development of this project, several technical decisions were made in order to ensure that the system performs reliably on financial time-series data. One of the most important decisions was the use of a sliding window walk-forward validation approach instead of a traditional train-test split or an expanding window. Financial data, especially cryptocurrency data is highly non-stationary. This means that often patterns change over time. Due to this, using older data may reduce the relevance of the model. The sliding window approach ensures that the model is always trained on more recent data even though it reduces the total amount of training data available at each step. This trade-off was considered acceptable because it improves the realism of the evaluation.

Another important decision was the choice of features used in the model. Instead of relying only on raw price values, additional features such as lagged prices, moving averages, and volatility measures were included. These features were chosen based on common practices in financial modelling as they help capture trends and short term patterns in the data. While more complex features could have been added, the focus was kept on widely understood indicators in order to maintain interpretability and reduce unnecessary complexity.

The selection of models also reflects a balance between simplicity and flexibility. Linear models such as OLS, Ridge, and Lasso were chosen as a baseline due to their interpretability and strong theoretical foundation. More complex models such as Kernel Ridge, K-Nearest Neighbours, and a neural network were included to explore whether non-linear methods could provide improvements. This allowed for a meaningful comparison between different levels of model complexity.

Finally, the decision to implement the algorithms from scratch using NumPy rather than relying on full external libraries was made to ensure a deeper understanding of the underlying mathematics. While this increased the development effort, it provided more control over the implementation and ensured transparency in how the models operate.

8. System Design

This section of the report focuses on how the final system was structured from a software engineering point of view. The codebase grew from a smaller regression based prediction model into a more complete application with multiple algorithms, multiple cryptocurrency pairs, and a graphical user interface as the project progressed and developed. Hence, a modular and object-oriented structure became important in order to keep the system understandable and maintainable.

The final system is organised into separate components for configuration, data loading, feature engineering, model implementation, tuning, evaluation, prediction, plotting, and GUI interaction. This modular structure allows each part of the system to focus on one specific responsibility rather than combining everything into one large script. This can be seen in the final code folder which includes files such as `config.py`, `data_loader.py`, `features.py`, `models_math.py`, `models_wrappers.py`, `tuning.py`, `evaluation.py`, `predictor.py`, `plotting.py`, and the GUI related files.

8.1 Design Objectives

Several main objectives guided the design of the system.

The first design objective was modularity. In order to make sure that the project stays organised even after more algorithms and GUI features were added, different stages of the workflow such as data loading, feature generation, model training, and prediction needed to remain separated.

The second design objective was maintainability. Since the project was growing and expanding as it developed it was important to make sure that modifications in one part of the code did not create unnecessary problems in another part.

The third design objective was extensibility. The final system was required to support more than just the initial regression algorithms on one dataset. It also had to support Kernel Ridge Regression, K-Nearest Neighbours, a feedforward neural network, 25 cryptocurrency pairs, and both console and GUI execution. This is visible in the configuration settings and tuning structure used across the project.

The fourth and final design objective was clarity. One of the key aims of the project was to understand how the algorithms work so it was important that the structure of the code remained transparent rather than hiding the full logic behind complete black-box implementations imported from external python libraries.

8.2 Overall Architecture

A pipeline structure is followed in the final system where data passes through several stages from collection to final output. The process begins at a high level with the user configurations and cryptocurrency selection done through the GUI. Then it moves through data loading, feature engineering, hyperparameter tuning, walk-forward evaluation, next-day prediction, and finally visualisation through plots and the GUI.

The class in this architecture are:

- Config: stores settings and tuning grids
- BTCDataLoader: handles historical market data retrieval
- FeatureEngineer” builds the processed dataset and features
- HyperparameterTuner: performs model tuning
- WalkForwardEvaluator: evaluates the models across rolling windows
- TomorrowPredictor: generates next-day forecasts
- Plotter: creates graphs
- BTCApp: manages the graphical interface

This separation of the stages of the code with each serving a distinct purpose allows the system to be easier to understand.

8.3 Main Components of the System

8.3.1 Configuration Component

config.py file focuses on the configuration of the system. This file contains the Config class which stores the default cryptocurrency pair, the range of years available to the user, the walk-forward training and test window sizes and the hyperparameter grids for the different models. It also includes structured data classes such as Dataset, BestParams and EvaluationResults which help organise how information moves between different parts of the program.

Since the same configuration object can be shared across the system rather than repeating settings in multiple files, this design improves consistency.

8.3.2 Data Loading Component

The class responsible for downloading historical cryptocurrency data from Yahoo Finance and checking whether enough historical data is available for the selected time range is the `BTCDataloader` class found in `data_loader.py`. It also handles insufficient data cases by raising an appropriate error. This means invalid datasets are filtered out before they enter the rest of the pipeline.

Separating this stage from the rest of the system is useful because data access is treated as its own responsibility rather than being mixed directly into training and prediction logic.

8.3.3 Feature Engineering Component

The `FeatureEngineer` class in `features.py` transforms raw market data into the structured inputs used by the models. These include lagged returns, moving averages, volatility indicators, RSI, EMA, MACD, momentum values, and other derived variables. The output of this part is a `Dataset` object containing the feature matrix, target values, prices, and other metadata required by the rest of the system.

This separation makes it easier to add or remove indicators during development and in the future.

8.3.4 Model Implementation Component

The mathematical logic of the prediction algorithms is contained in `models_math.py`. This includes the core implementations of OLS, Ridge Regression, Lasso Regression, Kernel Ridge Regression, K-Nearest Neighbours and the feedforward neural network. The regression related methods are all grouped under `RegressionMath` while the neural network logic is handled by `NeuralNetworkMath`.

The project also includes wrapper classes in `models_wrappers.py`. These include `RidgeClosedFormRegressor`, `LassoCDRegressor`, `KernelRidgeRegressor`, `KNNRegressor`, and `NeuralNetworkRegressor`. These wrappers provide a standard fit and predict interface so that the algorithms can be used consistently inside the tuning process.

This separation allows the mathematical implementation to stay separate from the higher level system workflow.

8.3.5 Tuning Component

The HyperparameterTuner class in tuning.py is the one responsible for selecting suitable hyperparameters for Ridge, Lasso, Kernel Ridge, KNN, and the neural network. It uses a time-series-aware tuning process and works with the model wrapper classes. Model selection remains separate from both historical evaluation and next-day prediction by keeping tuning in its own separate module.

This makes the design cleaner because hyperparameter tuning is treated as its own stage rather than being mixed directly into each algorithm.

8.3.6 Evaluation Component

The WalkForwardEvaluator class in evaluation.py performs the historical model performance testing of the system. It applies the sliding window walk-forward process in order to train the models repeatedly across time and store the results used later for graphs and comparison tables. This module also works with the Metrics class in metrics.py which provides Mean Squared Error, R^2 , and directional accuracy calculations.

This is one of the most important components because it produces the comparative results that are used in both the report and the GUI.

8.3.7 Prediction Component

The TomorrowPredictor class in predictor.py is responsible for generating next day forecasts using the most recent available training window. It first retunes the hyperparameters, then it retrains the algorithms on recent data, and finally it produces tomorrow's predicted return and price for each algorithm.

This component of the project connects all of the analytical side of the project with an actual user facing output i.e. the prediction of both price and return.

8.3.8 Plotting Component

The Plotter class in plotting.py generates the visual outputs used throughout the system. These include bar charts for return MSE, price MSE, and directional accuracy, as well as scatter plots comparing true and predicted returns and prices for each algorithm. These graphs are saved into the plots folder and then loaded into the GUI where needed.

By keeping the plotting separate from the rest of the system the visualisation logic is prevented from becoming mixed with data preparation or model training.

8.3.9 Interface Component

The system has two ways the user can utilize it: Graphical User Interface and Console version. The GUI is the main implementation and the preferred way to run the system. The console version is the basic default settings which were initially the foundation used to then create the GUI implementation. The console version is launched through `console_app.py` while the GUI is launched through `gui_app.py`. The GUI is built around the `BTCApp` class and is divided into several helper files such as `gui_prediction_tab.py`, `gui_performance_tab.py`, `gui_model_tabs.py`, `gui_helpers.py`, and `gui_styles.py`.

The graphical interface allows the user to select a cryptocurrency pair, choose the historical data window, switch between USD and GBP, run tomorrow predictions, load historical performance, and inspect model specific graphs.

8.4 Object-Oriented Design

The final system uses an object-oriented structure where the main stages of the workflow are represented by dedicated classes. This is different from the earlier proof-of-concept stage where the implementation was much smaller and more direct with everything in one file. The move to an object-oriented design reflects the development of the project from an early prototype into a more complete software engineering solution.

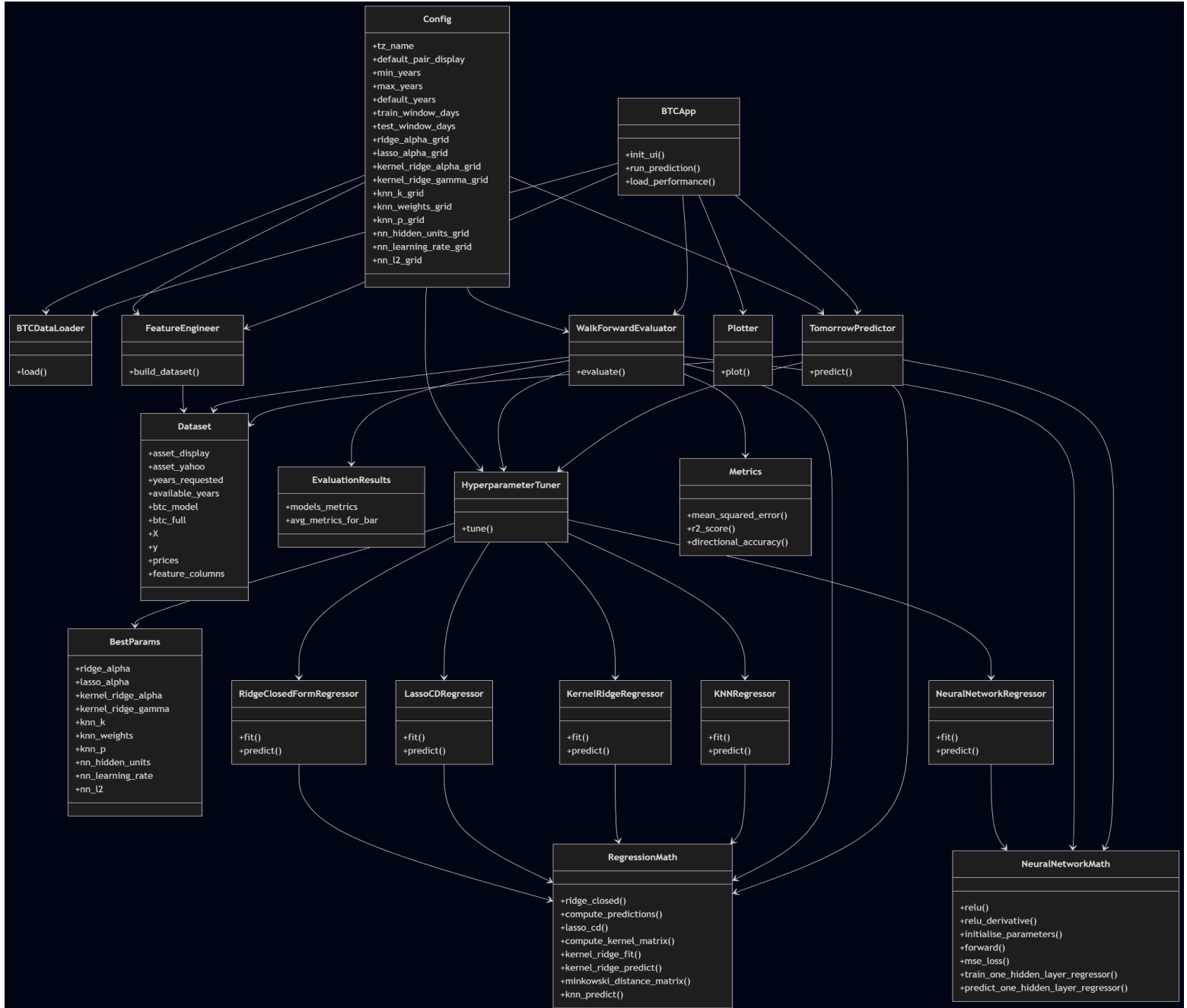
This design improves the project in several ways:

- it makes the code easier to follow
- it allows parts of the system to be reused
- it makes future extensions easier
- it supports debugging and testing more clearly

This became especially important once the project expanded to include more algorithms, more datasets, and a GUI because one file with everything made expanding and debugging the system very complicated and irritating.

8.5 Unified Modelling Language (UML) Diagram

The UML diagram for the final version of the system can be found below.



The purpose of this diagram is to show the relationships between the main classes of the project, including the configuration class, data loading class, feature engineering class, tuning class, evaluation class, prediction class, plotting class, and GUI class.

8.6 System Interaction Flow

The interaction between the parts of the system using the GUI implementation can be summarised as follows:

1. The user selects a cryptocurrency pair and a historical data window through the GUI application.
2. The data loader retrieves the historical data.
3. The feature engineering component creates the processed dataset and indicators.
4. The tuning component selects suitable hyperparameters.
5. The evaluation component performs historical walk-forward testing.
6. The prediction component generates next-day forecasts.
7. The plotting component saves the visual outputs.
8. The GUI loads and displays the final results.

The interaction between the main parts of the system using the console version can be summarised as follows:

1. The user runs the system using the console version which has set default input of BTC/USD data for a 10 year window.
2. The data loader retrieves the historical data.
3. The feature engineering component creates the processed dataset and indicators.
4. The tuning component selects suitable hyperparameters.
5. The evaluation component performs historical walk-forward testing.
6. The prediction component generates next-day forecasts.
7. The plotting component saves the visual outputs.
8. The console loads and displays the final results.

This interaction flow follows the same pipeline already mentioned in the methodology section but this time it is from a software design perspective rather than a methodological one.

8.7 Design Benefits

The final design provides the system with several practical benefits. Firstly, it makes the project easier to understand because each part of the system has a clear responsibility. Secondly, it makes the project easier to maintain since updates to one part usually do not require large changes elsewhere. Thirdly, it allows the project to scale more easily which became necessary once more algorithms and more cryptocurrency pairs were introduced. Finally, it gives the project a stronger software engineering foundation which is in accordance with the final year project expected deliverables. Overall, the final system design reflects the progression of the project from a smaller regression prototype into a more complete and modular cryptocurrency prediction framework.

9. Implementation

This part of the report explains how the system described in the previous sections was implemented in practice. The focus is mainly on how the different components were built in code and how they work together to produce the final output. While the methodology section explained the approach taken, this section focuses more on the actual implementation details of the system.

The implementation was done using Python through a modular and object-oriented structure. Each stage of the workflow such as data loading, feature engineering, model implementation, tuning, evaluation, and visualisation was implemented in a separate file. This made the code easier to manage and allowed different parts of the system to be developed and tested independently.

9.1 Overall Implementation Structure

The final codebase is organised into multiple files with each responsible for a specific part of the system. The main files include:

- `config.py`: stores configuration settings and data structures
- `data_loader.py`: retrieves and validates cryptocurrency data
- `features.py`: generates features from raw data
- `models_math.py`: contains mathematical implementations of algorithms
- `models_wrappers.py`: provides wrapper classes for model usage
- `tuning.py`: handles hyperparameter tuning
- `evaluation.py`: performs walk-forward validation
- `predictor.py`: generates next-day predictions
- `plotting.py`: creates and saves visualisations
- GUI files: handle user interaction

This separation follows the same modular design discussed in the system design section and ensures that each part of the system has a clear role.

9.2 Data Loading and Preprocessing Implementation

The data loading process is handled by the BTCDataLoader class. This class uses the yfinance library to download historical cryptocurrency data based on the selected pair and number of years.

When the data is retrieved the system then applies these preprocessing steps:

- The dataset is sorted chronologically
- Missing values are removed where necessary
- The dataset is validated to ensure sufficient data is available

This class also checks whether the requested time window is valid. If the dataset does not contain enough data, an error is raised. This prevents the system from continuing with invalid input. This ensures that only clean and reliable data is passed to the next stages of the pipeline.

9.3 Feature Engineering Implementation

Feature generation is handled by the FeatureEngineer class. This class takes the raw dataset and transforms it into a structured format suitable for modelling.

The main features implemented include:

- Lagged price values
- Moving averages
- Volatility measures
- Momentum indicators

The target variable is created by shifting the closing price forward by one day so that each row of features corresponds to the next day's price.

The output of this stage is stored in a Dataset object which contains:

- Feature matrix (X)
- Target values (y)
- Price data
- Feature names

This structured dataset is then used by the training, evaluation, and prediction components.

9.4 Algorithm Implementations

The core machine learning algorithms are implemented in `models_math.py`. These implementations are written using NumPy and do not rely on full external model implementations.

The basic regression based models used include:

- Ordinary Least Squares (OLS)
- Ridge Regression
- Lasso Regression

These are implemented using matrix operations and optimisation techniques such as coordinate descent for Lasso.

Additional models were implemented during a later stage of the project include:

- Kernel Ridge Regression
- K-Nearest Neighbours (KNN)
- Feedforward Neural Network

The KNN implementation uses a Minkowski distance calculation to determine nearest neighbours while the neural network is implemented as a small feedforward network with one hidden layer. To integrate these models into the system, wrapper classes are used in `models_wrappers.py`. These wrappers provide a consistent interface with `fit()` and `predict()` methods thus allowing all models to be used in the same way during tuning and evaluation.

9.5 Hyperparameter Tuning Implementation

Hyperparameter tuning is handled by the `HyperparameterTuner` class. This class tests different combinations of parameters for each model and selects the ones that produce the best performance.

Examples of these tuned parameters include:

- Alpha values for Ridge and Lasso
- Gamma values for Kernel Ridge
- Number of neighbours for KNN
- Hidden units and learning rate for the neural network

The tuning process is performed using a time-series-aware approach rather than random splitting. This ensures that the selected parameters are suitable for real world prediction conditions.

9.6 Walk-Forward Evaluation Implementation

The evaluation process is implemented in the WalkForwardEvaluator class. This class performs the sliding window validation described earlier.

The implementation for this works in the following way:

1. Selecting a fixed training window
2. Training the model
3. Testing on the next period
4. Moving the window forward
5. Repeating the process

This is done for all algorithms and across the full dataset. The performance of each model is stored and later used for comparison. The evaluation also makes use of the Metrics class which calculates:

- Mean Squared Error (MSE)
- Directional accuracy
- R^2 score

These metrics are stored in structured dictionaries and later visualised using graphs.

9.7 Prediction Implementation

The next day prediction is handled by the TomorrowPredictor class.

This component does the following things:

- It uses the most recent training window
- It retunes the hyperparameters
- It retrains each model
- It generates predictions for the next day

The output includes:

- Predicted return
- Predicted price
- Model-specific results

This part of the system is important because it provides a real world use case where you actually get a price prediction which can possibly be used as a guide to making a trade rather than just historical evaluation of the different algorithms.

9.8 Visualisation Implementation

The visualisation of results is handled by the Plotter class in plotting.py.

The system generates the following things:

- Bar charts comparing MSE and directional accuracy
- Scatter plots for true vs predicted values
- Model comparison graphs

These plots are saved to a folder and later displayed in the GUI.

The visual outputs make it easier to compare the performance of different models and identify patterns in their behaviour.

9.9 Graphical User Interface (GUI)

A graphical user interface was implemented to improve usability and make the system easier to interact with for the user.

9.9.1 GUI Description

The GUI is built using Python and is organised into multiple files to keep it structured. Here are the files used for the GUI and a small description for them:

- `gui_app.py`: this file contains the main application
- `gui_prediction_tab.py`: this file contains the next day prediction interface
- `gui_performance_tab.py`: this file contains the historical performance comparison
- `gui_model_tabs.py`: this file contains the model specific plots
- `gui_helpers.py`: this file contains the helper functions
- `gui_styles.py`: this file contains the styling for GUI

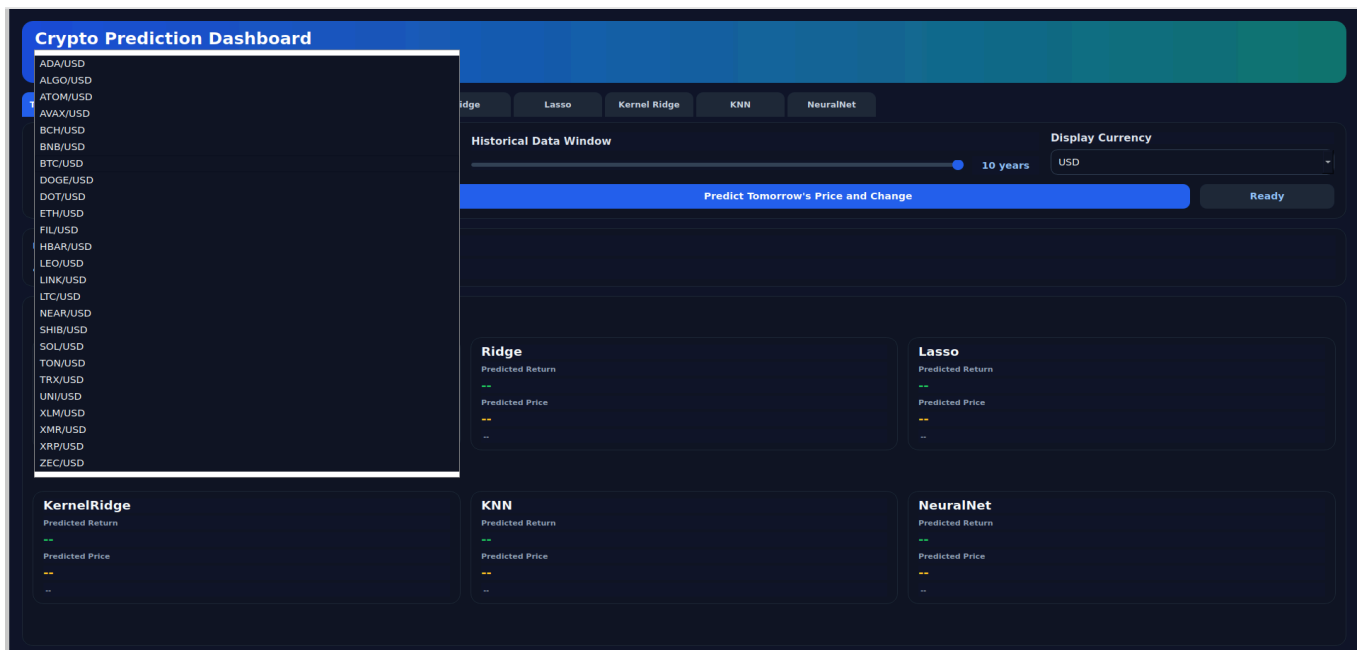
The interface allows the user to do the following:

- Select cryptocurrency pairs
- Choose the amount of data
- Switch between USD and GBP
- Run predictions
- View performance comparisons based on historical data
- View model specific performances

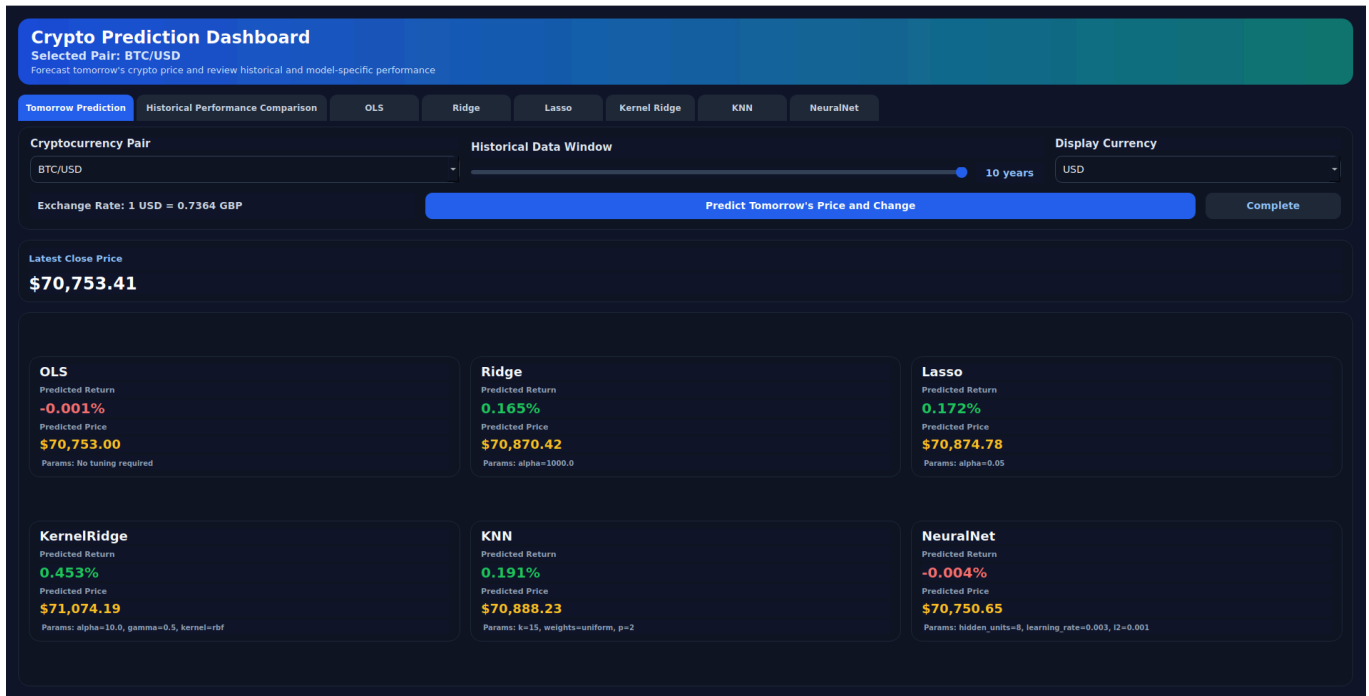
9.9.2 GUI Visuals

In this part of the report GUI visuals can be seen for a demo run of the model to see how it works.

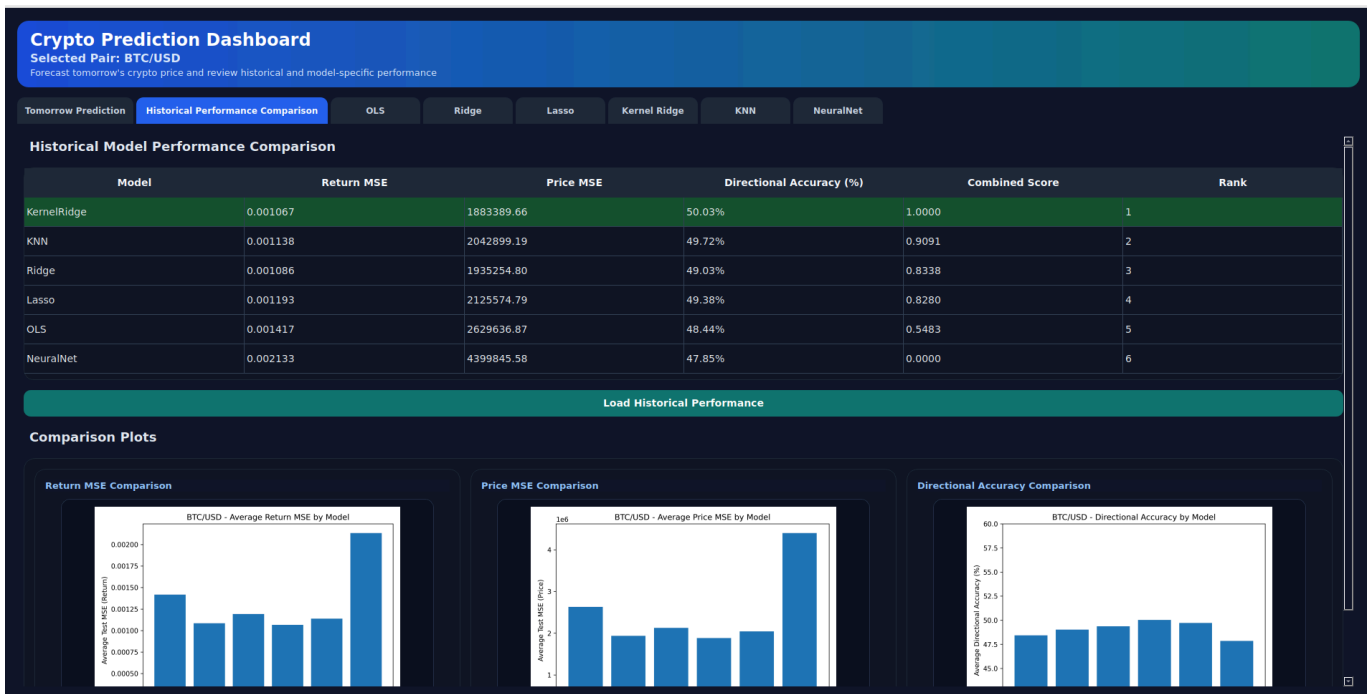
The Main Starting Tab/ Prediction Tab of the GUI



Prediction Tab of the GUI after Model Run for BTC/USD data of 10 years using USD



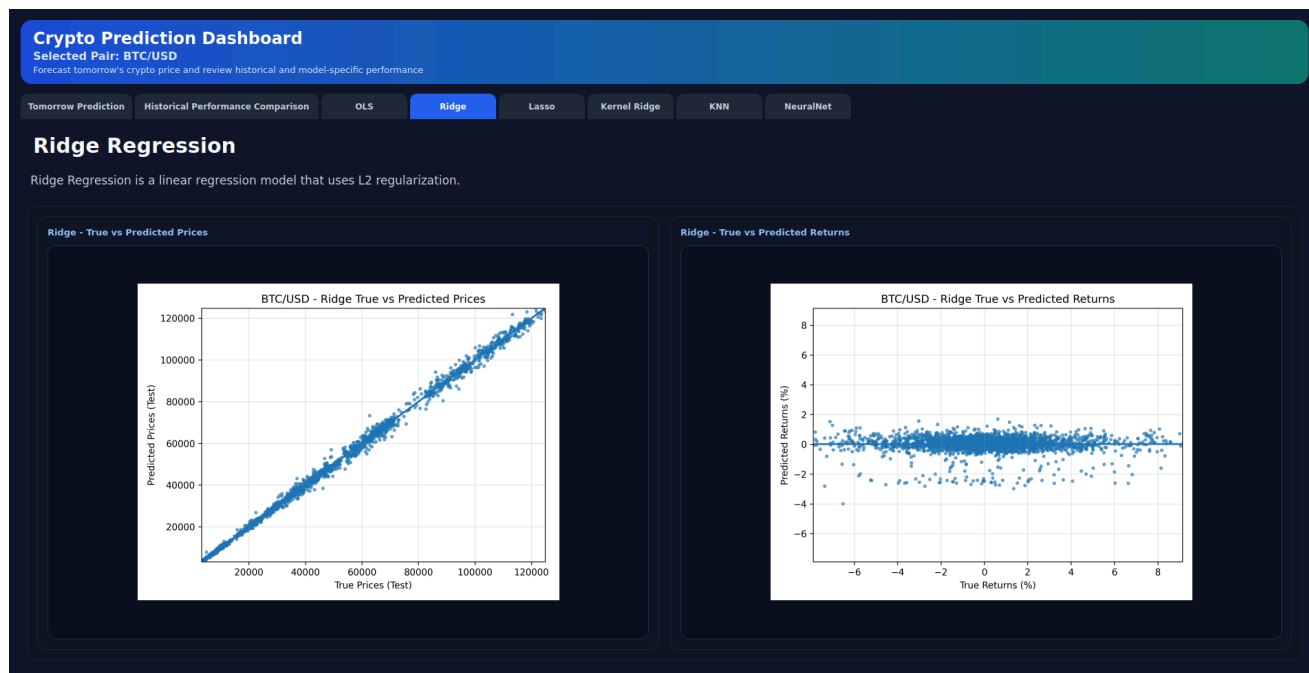
Historical Model Performance Tab for this Run



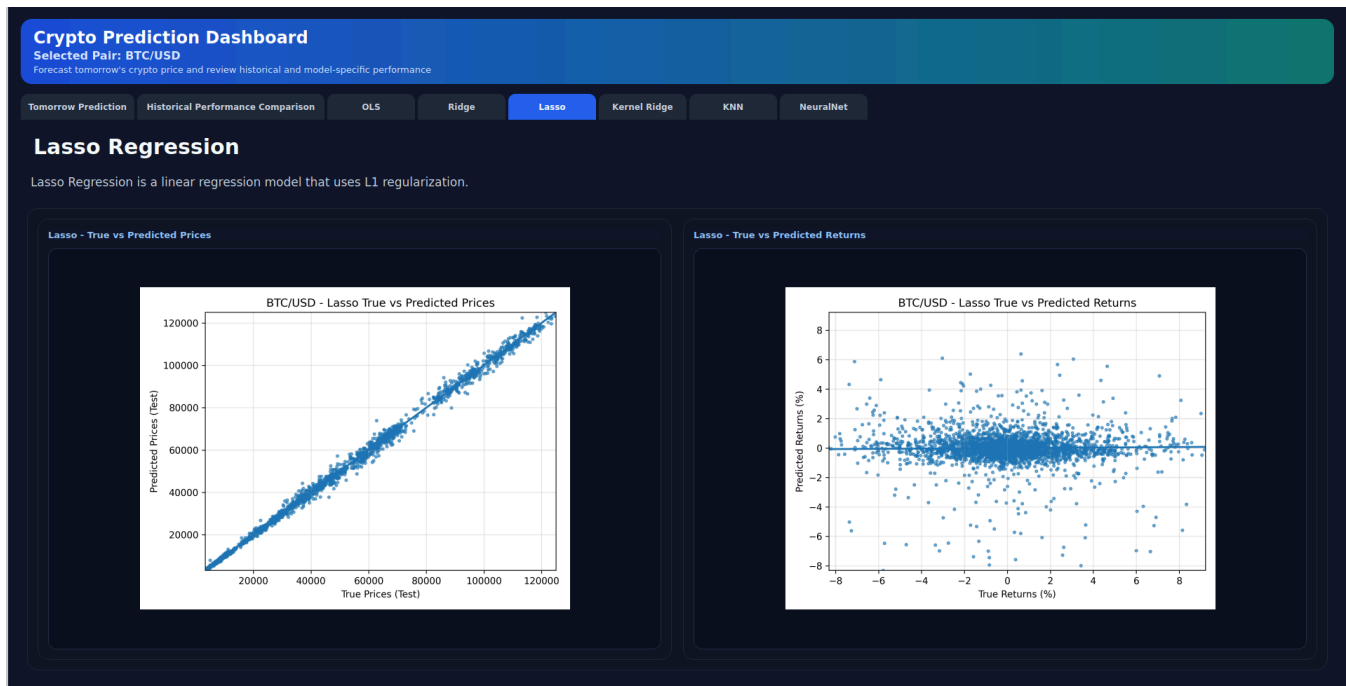
OLS Regression Algorithm Tab for this Model Run



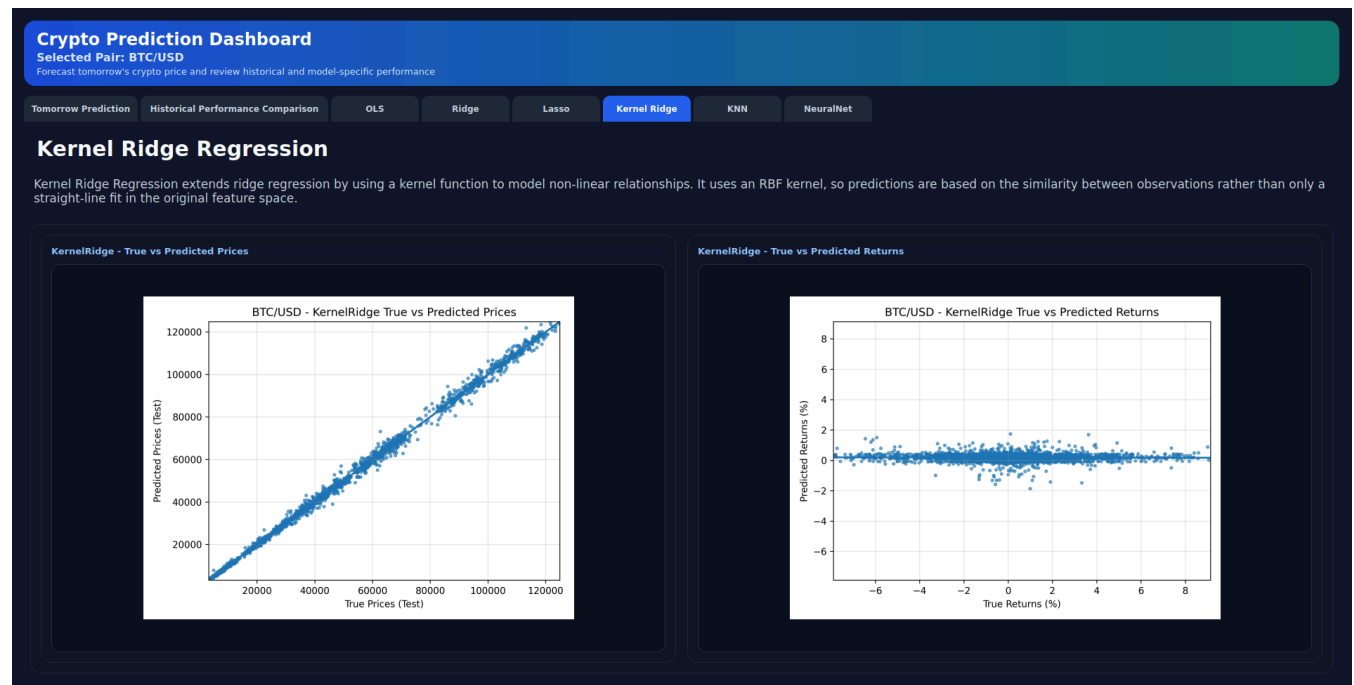
Ridge Regression Algorithm Tab for this Run



Lasso Regression Algorithm Tab for this Run



Kernel Ridge Regression Algorithm Tab for this Run



K-Nearest Neighbour Algorithm Tab for this Run



Feedforward Neural Network Algorithm Tab for this Run



9.10 Implementation Summary

The implementation overall successfully transforms the theoretical concepts and methodology into a working system. The modular structure makes sure that each component can be developed and tested independently while still contributing to the overall pipeline.

The use of object-oriented design makes the system easier to extend and maintain, which was important as the project expanded and progressed to include more algorithms, datasets, and user interface features.

The final implementation demonstrates a complete workflow from data collection to prediction and visualisation thus providing both analytical insights and a user-friendly interface. This demonstrates that the system is not only theoretically grounded but also practically usable.

9.11 Development Process and Challenges

During the development of this project, the system evolved from a simple proof-of-concept into a more complex application with multiple components and a graphical user interface. This required an iterative development process where features were gradually added and tested. Initially, the focus was on implementing the regression models correctly using synthetic data. Once this was achieved, the system was extended to handle real world cryptocurrency data and additional models.

Several practical challenges were encountered during development. One challenge was ensuring that the system worked consistently across different environments, particularly when handling file paths for saving plots and running the graphical interface. Debugging these issues required careful inspection of how directories were handled within the code. Another challenge was maintaining consistency between different components of the system as the project grew. This was addressed by adopting a modular design where each part of the system had a clearly defined responsibility.

Testing was performed throughout development to ensure that individual components worked as expected. This included verifying that data was loaded correctly, checking that feature engineering produced valid outputs, and confirming that models generated reasonable predictions. While a formal automated testing framework was only implemented to do the checking of the code logic, manual testing was used to validate the behaviour of the system at each stage.

Overall, the development process involved continuous refinement and debugging, with improvements being made as new issues were identified. This iterative approach helped ensure that the final system was both functional and stable.

10. Results

This section of the report presents the results from the different algorithms implemented in the system. The results are based on the walk-forward evaluation approach described earlier. In this approach models are trained on past data and tested on unseen future data.

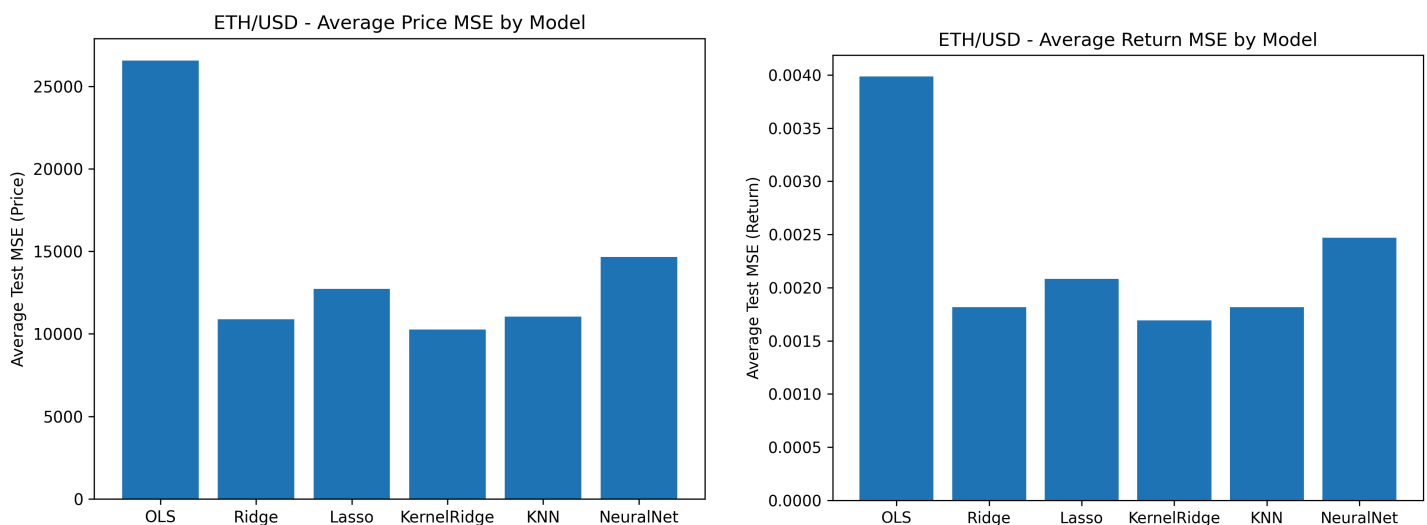
The performance of the models is evaluated using the following metrics:

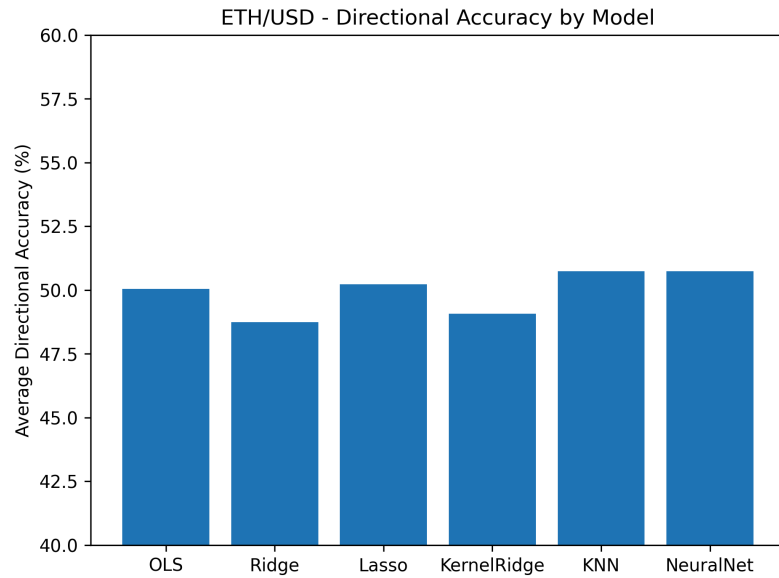
- Mean Squared Error (MSE) on returns
- Mean Squared Error (MSE) on price
- Directional Accuracy

These metrics provide different perspectives on model performance. MSE measures how close the predictions are numerically. On the other hand directional accuracy evaluates whether the model correctly predicts the direction of price movement.

Due to the large number of cryptocurrency pairs analysed, only a selection of representative results is shown below. These examples were chosen to highlight different market behaviours while maintaining clarity in presentation.

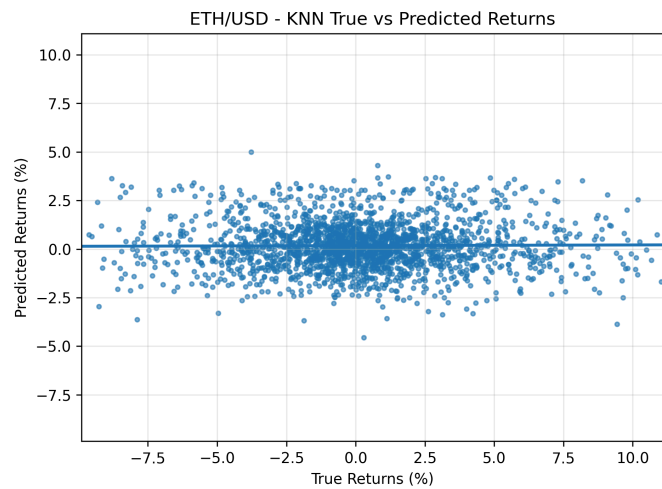
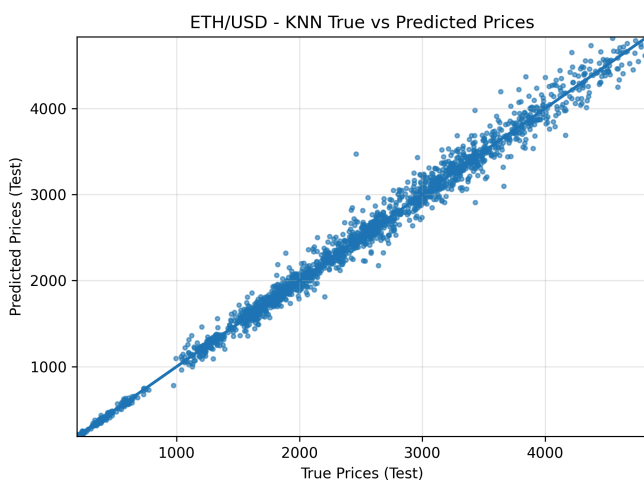
10.1 Case Study 1: Ethereum (ETH)





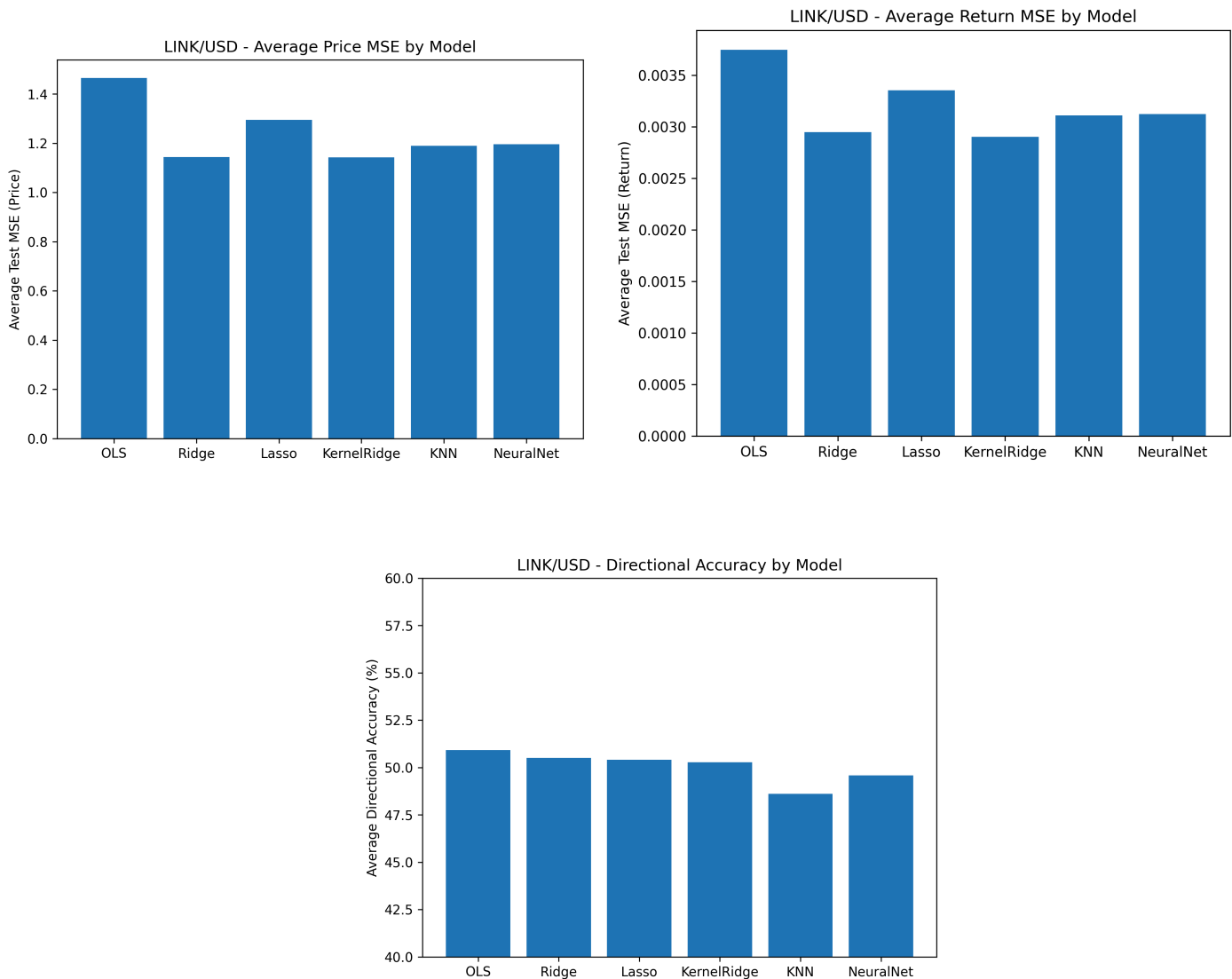
Ethereum represents a major cryptocurrency with relatively high liquidity and more stable behaviour compared to smaller assets.

From these results, it becomes evident that model performance is relatively consistent across metrics. In this case the best performing algorithm was K-Nearest Neighbour. Below are the price and return true vs predicted scatterplots for K-Nearest Neighbour algorithm in this instance. In the case of price scatterplot the reference line is an ideal line while in the case of return scatterplots the reference line is the line of best fit. This is relevant to all scatterplots in the system.



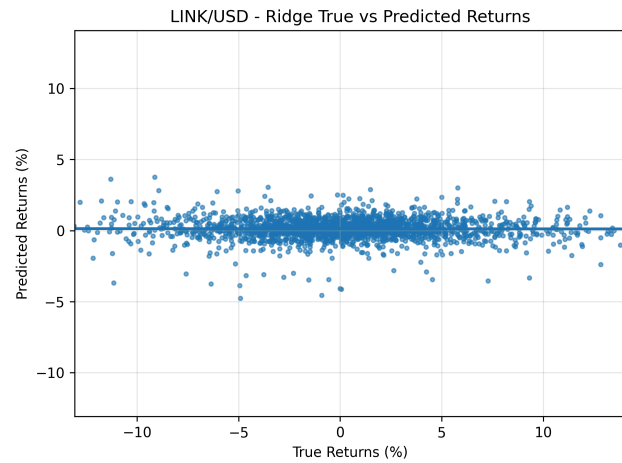
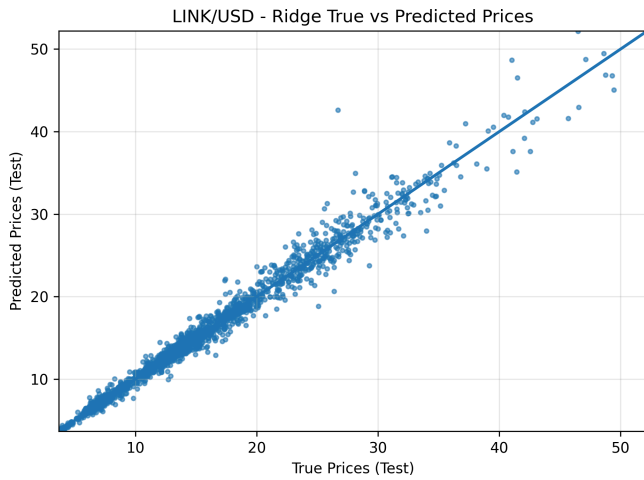
The scatter plots show predictions closer to the ideal line in this case. This suggests that more stable market conditions allow models to better capture underlying patterns.

10.2 Case Study 2: Chainlink (LINK)



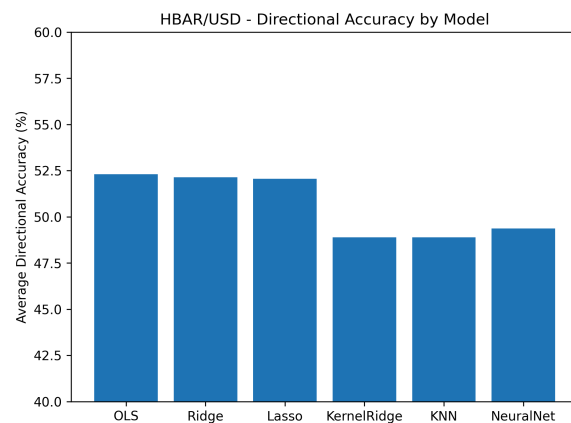
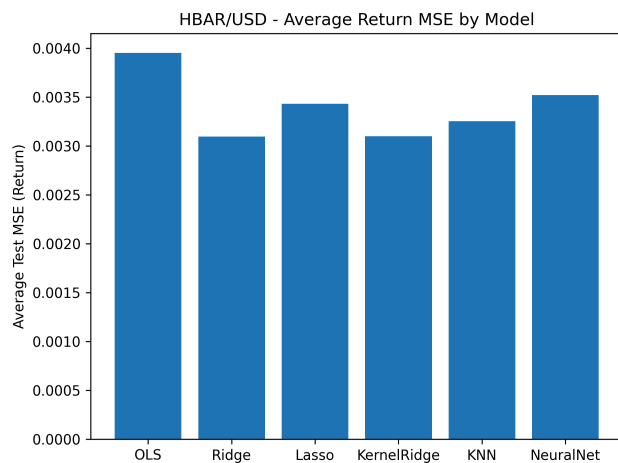
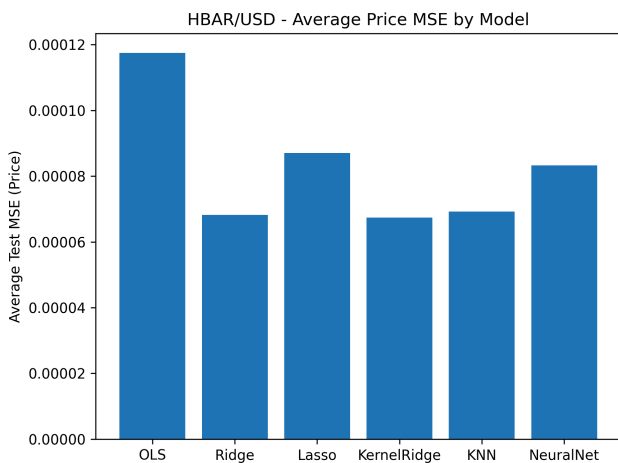
Chainlink represents a mid-level cryptocurrency with moderate volatility.

From these results, it becomes evident that the model performance across metrics varies a little bit more here as compared to Ethereum. An example of this is that OLS has the worst price and return MSEs however it has the best Directional Accuracy. In this case the overall best performing algorithm was Ridge Regression. Below are the price and return true vs predicted scatterplots for Ridge Regression algorithm in this instance.



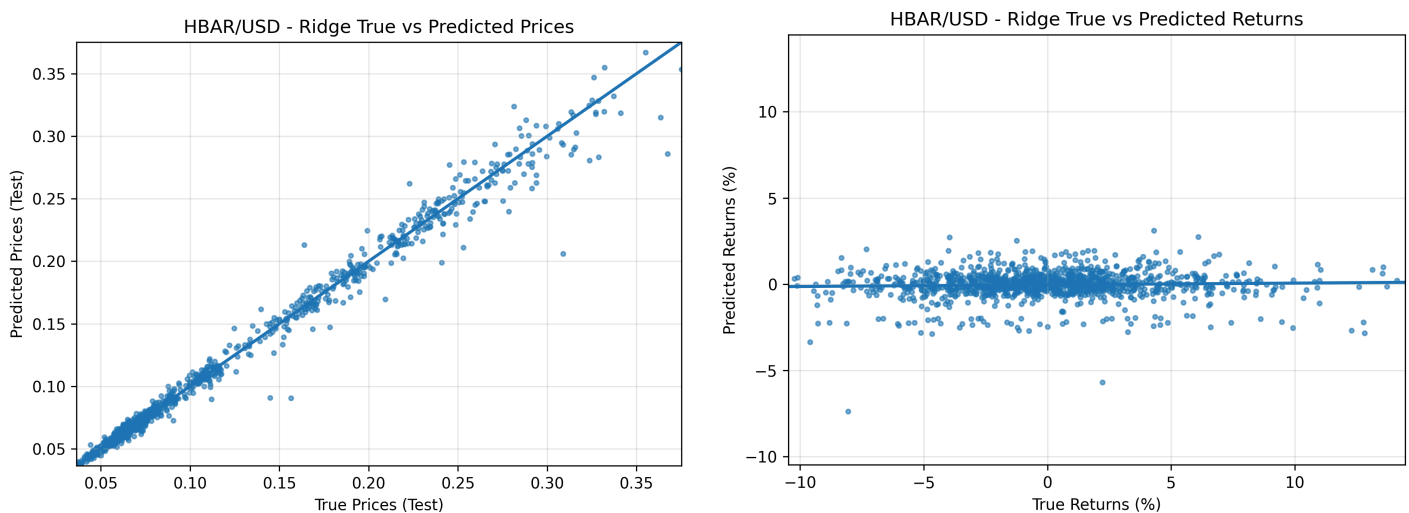
The scatter plots show more spread in predictions. This indicates that increased volatility makes it harder for models to maintain both accuracy and consistency.

10.3 Case Study 3: HBAR



HBAR represents a more volatile cryptocurrency with less stable price behaviour.

From these results, it becomes evident that the model performance across metrics varies quite a bit where some algorithms perform better in MSEs for price and return while the others perform better in Directional Accuracy. An example is Lasso Regression whose performance is amongst the worst for both MSEs however it is among the best for directional accuracy. Overall the best performing algorithm in this case is Ridge Regression. Below are the price and return true vs predicted scatterplots for Ridge Regression algorithm in this instance.



The scatter plots show greater dispersion away from the ideal line as compared to the other two cases. This reflects the difficulty of predicting assets with higher volatility and less predictable patterns.

10.4 Summary of Results

Overall, the results show that all models are able to capture some patterns in the data, There is no one single model which consistently outperforms the others across all datasets. The performance of the algorithms vary depending on the cryptocurrency being analysed.

Other observations that are noticeable are that more stable assets tend to produce more reliable predictions while the more volatile assets lead to higher prediction errors. Finally, it can be observed that different models perform better under different conditions.

11. Discussion

This section of the report discusses the results previously presented in the report. It attempts to explain the behavior observed from the different algorithms. The aim is to understand why certain models performed better under specific conditions and how these results relate to the nature of financial time series data.

11.1 Impact of Market Volatility

The effect of market volatility on model performance is one of the clearest observations from the results. For cryptocurrency like Ethereum which is a more stable and highly traded cryptocurrency, the models produced more consistent and accurate predictions. In contrast, for assets such as HBAR which exhibit higher volatility the performance of all models decreased significantly. This suggests that model performance is strongly influenced by the stability of the underlying data. Patterns are more consistent and easier to learn in more stable markets whereas in highly volatile markets, price movements become more unpredictable thus making accurate prediction more difficult. This aligns with the general understanding of financial markets where increased volatility often reduces predictability because it makes it harder to recognize a pattern.

This behaviour can also be linked to the concept of non-stationarity in financial time-series data. As market conditions change over time, the statistical properties of the data, such as mean and variance, may also change. This makes it difficult for models to learn stable patterns that remain valid in the future. As a result, even models which generally perform well may struggle when applied to new unseen data, especially in highly volatile markets.

11.2 Comparison Between Linear and Non-Linear Models

The results show that linear models such as OLS, Ridge, and Lasso perform competitively across most datasets despite their relatively simple structure.

Ridge Regression in particular often performs slightly better than OLS. This can be explained by the use of regularisation which reduces overfitting and helps the model generalise better to unseen data. This is especially important in financial datasets where you commonly find noise in the data.

Non-linear models such as Kernel Ridge Regression and the neural network were expected to outperform linear models due to their ability to capture more complex relationships. However, the results show that this is not always the case.

A possible explanation for this behaviour is that cryptocurrency data contains a high level of noise relative to the underlying signal. Simpler linear models in such cases can perform just as well as, or sometimes better than, more complex models because they are less likely to overfit short-term fluctuations. More flexible models such as neural networks have a greater capacity to learn complex relationships, but when the data is highly noisy and limited in size, this additional flexibility can lead to poorer generalisation. This highlights the importance of considering the nature of the dataset when selecting an appropriate model rather than assuming that more complex methods will always perform better.

11.3 Behaviour of K-Nearest Neighbours

The KNN model showed mixed performance across different datasets.

In some cases such as Ethereum, it performed relatively well. In particular in the directional accuracy metric. This may be because KNN relies on identifying similar past patterns which can be effective when market behaviour is more stable.

However in more volatile datasets, KNN performance was less consistent. Since KNN does not build a generalised model and instead relies directly on past observations, it can struggle when market conditions change and past patterns are no longer relevant.

This highlights a limitation of instance-based learning methods when applied to financial data.

11.4 Relationship Between MSE and Directional Accuracy

An important observation from the results is that Mean Squared Error and directional accuracy do not always align.

Some models achieved lower MSE but did not necessarily achieve the highest directional accuracy. While others showed the opposite behaviour. This indicates that predicting the exact numerical value of a price is different from predicting its direction. Directional accuracy can sometimes be more relevant in financial applications, as correctly predicting whether the market will move up or down may be more useful than predicting the exact value.

This highlights the importance of using multiple evaluation metrics when assessing model performance.

11.5 Limitations of Neural Networks in This Context

The neural network model which can be considered more flexible did not consistently outperform simpler models. There may be several reasons why this was the case.

Firstly, the dataset size might not have been big enough to fully train a neural network. Secondly, the neural network used in this project is relatively simple, consisting of a single hidden layer which may limit its ability to capture more complex patterns in the data. Also, financial data contains a high level of noise which makes it difficult to capture the inner patterns.

As a result, the neural network may struggle to learn stable patterns and may instead fit to short-term fluctuations. This demonstrates that more complex models do not always lead to better performance. In particular in cases where data is noisy and unpredictable.

11.6 Effectiveness of Walk-Forward Validation

The use of walk-forward validation provides a more realistic evaluation of model performance compared to traditional train-test splits.

By training on past data and testing on future data, the evaluation process better reflects real world prediction scenarios. This ensures that models are not unintentionally using future information which would lead to overly optimistic results. The results obtained through this approach show more variability compared to standard validation methods but they provide a more reliable measure of how the models would perform in practice.

11.7 Overall Interpretation

Overall, the results suggest that predicting the next day price for cryptocurrencies remains a challenging task. While the models are able to capture some patterns in the data, their performance is limited by factors such as market volatility, noise in the data and changing market conditions.

The findings also show that simpler models such as Ridge Regression can perform competitively, more complex models do not guarantee better results and model performance depends heavily on the dataset.

This reinforces the idea that no single model is universally best for financial prediction.

12. Evaluation

This section of the report evaluates the overall performance of the system while considering both the technical implementation and the results obtained. The aim is to assess how well the system meets the objectives set out earlier and how effective it is in practice.

12.1 Achievement of Objectives

The main objective of the project was to implement ridge regression and apply it to real-world data while also comparing it with other machine learning models. This objective has been achieved. Ridge regression was successfully implemented from scratch and tested on multiple cryptocurrency datasets. In addition, other models such as Lasso, Kernel Ridge, K-Nearest Neighbours, and a neural network were also implemented from scratch and compared with each other based on their performance.

The system was also extended beyond the initial scope by including multiple cryptocurrency pairs, additional features and a graphical user interface. This shows that the project not only met its original aims but also went further in terms of functionality.

12.2 Model Performance

The models implemented in the system were able to capture some patterns in the data although their performance varied depending on the cryptocurrency being analysed. Regression based models such as Ridge regression performed consistently well and provided a strong baseline. More complex models such as Kernel Ridge and neural networks showed potential but did not consistently outperform simpler approaches. This suggests that while more advanced models can offer improvements, their performance is limited by the nature of financial data which is often noisy and unpredictable.

It is also important to note that the choice of evaluation metrics influences how model performance is interpreted. Mean Squared Error focuses on numerical accuracy, while directional accuracy reflects the ability to predict market movement. Therefore, using multiple metrics provides a more complete understanding of model performance rather than relying on a single measure.

12.3 Effectiveness of the System

From a system perspective, the implementation works as intended. The data can be loaded and processed correctly. The models can be trained and evaluated correctly. The results are visualised through plots and finally the GUI allows for user interaction with the system. The system provides a complete pipeline from data collection to prediction and visualisation. This makes it usable as a practical tool rather than just a theoretical implementation.

12.4 Strengths of the Approach

There are several strengths in the approach taken in this project.

The use of walk-forward validation provides a realistic evaluation of model performance because it prevents leakage of future data to the model thus making the results more accurate.

Multiple models are compared rather than relying on a single approach. This allows for an insight into how the performances of different algorithms vary on the same dataset.

The system is modular and organised thus making it easier to not only extend it but also in case of debugging it allows for changes to be made to the relevant parts more easily without affecting the full code.

The inclusion of a GUI improves usability and presentation. It allows the user to interact with the system and be able to make choices regarding the data used by the system.

These aspects contribute to both the technical quality of the project and its overall usability.

12.5 Areas for Improvement

While the system performs well overall, as always there are some areas that could be improved. The neural network model is relatively simple and could be expanded. Also, the system relies only on historical price-based features. Furthermore, the performance is limited by the quality and nature of the data. These limitations do not prevent the system from functioning, but they do highlight areas where further improvements could be made.

12.6 Summary

Overall, the system successfully meets the objectives of the project and demonstrates a working implementation of multiple regression based and machine learning models applied to financial data. While the results show that predicting cryptocurrency prices is challenging, the system provides useful insights into model behaviour and highlights the strengths and weaknesses of different approaches.

13. Limitations

This section outlines the main limitations of the system developed in this project. While the system performs as intended and provides useful insights there are several factors that restrict its overall performance and applicability.

13.1 Data Limitations

One of the main limitations of this project is the nature of the data used. The system relies entirely on historical price data obtained from Yahoo Finance. While this data is sufficient for building and testing models, it does not capture many external factors that influence cryptocurrency markets such as news events, macroeconomic conditions or market sentiment. Additionally, although multiple years of data were used, financial time-series data is often non-stationary. This means that patterns observed in the past may not hold in the future which limits the reliability of predictions.

13.2 Model Limitations

Another limitation in this system is related to the models themselves. The regression models used in this project assume that relationships between features and the target variable remain relatively stable. However, in highly volatile markets such as cryptocurrency these relationships can change rapidly. The neural network model is also relatively simple consisting of a single hidden layer. While this was sufficient for the current scope of the project, it may not be powerful enough to capture more complex patterns in the data. Similarly, K-Nearest Neighbours relies heavily on past observations and may struggle when market conditions shift as previously seen in the results.

13.3 Feature Engineering Limitations

The system primarily uses technical indicators derived from historical price data such as lag values, moving averages, and volatility measures. While these features are commonly used in financial modelling, they may not fully capture the underlying drivers of price movements. Important information such as trading volume dynamics, order book data, or external market signals is not included. This limits the ability of the models to make more informed predictions.

13.4 Evaluation Limitations

Although walk-forward validation provides a more realistic evaluation compared to traditional methods, it still has limitations. The choice of window size can influence results and different configurations may lead to different performance outcomes. Additionally, the evaluation focuses on next day prediction which is only one of the possible forecasts you can make. The conclusions drawn from the evaluation as a result are limited to this specific setup.

13.5 System Limitations

From a system perspective, the implementation is designed for experimentation rather than real world trading. The system does not account for transaction costs, slippage or execution delays which would all affect the practical usefulness of the predictions. Furthermore, the GUI improves usability, however it is mainly designed for demonstration purposes and could be further improved in terms of robustness and scalability.

13.6 Summary

Overall, the system demonstrates a working application of regression based and machine learning models to cryptocurrency prediction. However, its performance is constrained by the limitations of the data, models, and evaluation approach. These limitations highlight the challenges involved in financial time series prediction and provide a basis for potential improvements in future work.

14. Future Work

This section of the report looks into potential improvements and extensions that could be made to the system in the future. These ideas build directly on the limitations identified in the previous section and aim to improve both the performance and practicality of the model.

14.1 Incorporation of Additional Data Sources

One possible improvement would be to include additional data beyond historical price values. In this project, the models rely mainly on technical indicators derived from price data. However, cryptocurrency markets are influenced by many external factors such as news events, social media sentiment, and macroeconomic conditions. In future work, incorporating sentiment analysis or news based features could improve the model's ability to capture real world influences on price movements. Maybe one of the features future works could look into is the google trend of the cryptocurrency and a higher trend in searches could hint towards a higher trade volume in that cryptocurrency.

14.2 Improvement of Neural Network Architecture

The neural network used in this project is relatively simple and consists of a single hidden layer. Future work could involve experimenting with deeper neural network architectures or more advanced models such as recurrent neural networks (RNNs) or Long Short-Term Memory (LSTM) networks. These models are specifically designed for time-series data and may be better suited for capturing temporal dependencies in financial datasets. However, this would also require larger datasets and careful tuning to avoid overfitting.

14.3 Expansion of Feature Engineering

Another area for improvement is feature engineering. The current system uses common technical indicators such as lag values, moving averages, and volatility measures. In future works additional features could be used. These could include more advanced indicators or combinations of existing ones. Improving feature representation may allow the models to capture more meaningful patterns in the data.

14.4 Alternative Evaluation Strategies

Although walk-forward validation provides a realistic evaluation framework, alternative approaches could also be explored. Some examples of this would be testing using different window sizes or forecasting horizons such as multi-day predictions. This would provide a broader understanding of how model performance changes under different conditions.

14.5 Real World Trading Considerations

The current system focuses on prediction rather than practical trading. Future work could involve extending the system to include trading strategies based on the predictions. This would require considering factors such as transaction costs, risk management, and portfolio allocation. An extension like this would make the system more applicable to real world financial decision making.

14.6 System Improvement

From a software engineering perspective, the system could be further improved in terms of scalability and robustness. An example of this would be enhancing the GUI to be able to handle larger datasets more efficiently and provide additional visualisations. The system could also be extended to support real time data updates rather than relying solely on historical data.

14.7 Summary

Overall, there are several ways in which the system could be extended and improved. These include incorporating additional data sources, improving model complexity, expanding feature engineering, and making the system more suitable for real world applications. These potential improvements highlight that while the current system provides a strong foundation, there is still significant scope for further development.

15. Conclusion

This project set out to investigate the use of regression based machine learning algorithms for predicting cryptocurrency price movements. The aim was not only to understand the theoretical foundations of these models but also to evaluate how they perform when applied to real world financial time-series data.

A range of regression models including Ordinary Least Squares, Ridge Regression, and Lasso Regression were implemented from scratch and applied to cryptocurrency datasets. The project was then extended to include additional algorithms such as Kernel Ridge Regression, K-Nearest Neighbours and a feedforward neural network in order to allow for a broader comparison.

The system developed throughout this project provides a complete pipeline, starting from data collection and preprocessing, moving through feature engineering and model training, and ending with evaluation, visualisation, and next-day prediction. The use of walk-forward validation ensured that the evaluation process reflected realistic prediction scenarios.

The results showed that while all models were able to capture some patterns in the data, no single model consistently outperformed the others across all datasets. Simpler models such as Ridge Regression often performed competitively with more complex approaches, particularly in stable market conditions. In contrast, more volatile assets proved significantly harder to predict, highlighting the challenges associated with financial time-series modelling.

Overall, the findings suggest that while regression based models provide a strong and interpretable baseline for financial prediction, their performance is limited by the noise, volatility, and non-stationary nature of cryptocurrency markets. More complex models do not necessarily guarantee better results, especially when the underlying data is unpredictable.

In conclusion, this project demonstrates both the potential and the limitations of machine learning approaches in financial prediction. It highlights the importance of model selection, feature engineering, and validation strategies, while also showing that the nature of the data plays a crucial role in determining model performance.

16. Bibliography

1. Bishop, C. M. (2006). Pattern recognition and machine learning. Springer.
2. Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221717310652>
3. Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020) ‘Array programming with NumPy’, *Nature*, 585, pp. 357–362.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer.
5. Hunter, J.D. (2007) ‘Matplotlib: A 2D Graphics Environment’, *Computing in Science & Engineering*, 9(3), pp. 90–95.
6. McKinney, W. (2010) Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, pp. 51–56.
7. Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert Systems with Applications*, 42(1), 259–268. <https://www.sciencedirect.com/science/article/abs/pii/S0957417414004473>
8. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
9. Riverbank Computing Limited (2024) PyQt5 documentation. Available at:
<https://www.riverbankcomputing.com/software/pyqt/>
10. Yahoo Finance (2024) Bitcoin USD (BTC-USD) Historical Data. Available at:
<https://finance.yahoo.com/quote/BTC-USD/history/>

17. Appendices

17.1 Appendix A

The list of all 25 cryptocurrency pairs is as follows:

1. ADA/USD (Cardano)
2. ALGO/USD (Algorand)
3. ATOM/USD (Cosmos)
4. AVAX/USD (Avalanche)
5. BCH/USD (Bitcoin Cash)
6. BNB/USD (Binance Coin)
7. BTC/USD (Bitcoin)
8. DOGE/USD (Doge Coin)
9. DOT/USD (Polkadot)
10. ETH/USD (Ethereum)
11. FIL/USD (File coin)
12. HBAR/USD (Hedera Hashgraph)
13. LEO/USD (UNUS SED LEO)
14. LINK/USD (Chainlink)
15. LTC/USD (Litecoin)
16. NEAR/USD (NEAR Protocol)
17. SHIB/USD (Shiba Inu)
18. SOL/USD (Solana)
19. TON/USD (Telegram Open Network)
20. TRX/USD (TRONix)
21. UNI/USD (Uniswap)
22. XLM/USD (Stellar Lumens)
23. XMR/USD (Monero)
24. XRP/USD (Ripple)
25. ZEC/USD (ZCash)

